

Chapter-wise Summary of Database Management Systems

Unit I: Chapter 1 – Introduction to Database Management Systems and ER Model

This chapter covers the fundamentals of database systems, data models, structure, languages, and the Entity-Relationship (ER) modeling approach.

Important Points to Remember:

1. **DBMS Definition and Purpose:** A DBMS is a collection of interrelated data and programs used to store and retrieve information efficiently. The purpose is to manage commercial data previously stored in file processing systems.
2. **Advantages of DBMS:** DBMS removes data redundancy, allows efficient data retrieval, maintains data integrity and atomicity, supports concurrent access, and enforces security policies.
3. **Data Abstraction Levels:** There are three levels: **Physical level** (describes how data is stored, lowest level), **Logical level** (describes what data is stored and relationships), and **View level** (highest level, describes only part of the database for specific users).
4. **Database Languages:** These include **DDL** (Data Definition Language, for defining schema like CREATE, DROP), **DML** (Data Manipulation Language, for accessing/manipulating data like SELECT, INSERT), and **DCL** (Data Control Language, for managing access/authorization like GRANT, REVOKE).
5. **Data Models:** Conceptual tools for describing data and relationships, including Relational Model, Entity-Relationship (E-R) Model, Object-Based Data Model, Semi-structured data model, Hierarchical Model, and Network Model. The E-R model is widely used in database design.
6. **Data Independence:** The ability to modify the schema at one level (e.g., physical or conceptual) without requiring changes at the next higher level.
7. **ER Model Concepts:** Entities (objects distinguishable from others), Attributes (properties of entities, e.g., composite, multivalued, derived), Relationships (associations among entities), and Keys (used to uniquely identify entities).
8. **Constraints in ER:** Structural constraints limit participation, commonly represented by **Cardinality Ratio** (One-to-One, One-to-Many, Many-to-One, Many-to-Many).
9. **Extended ER Features (EER):** Include **Specialization** (top-down creation of subclasses), **Generalization** (bottom-up creation of superclasses), and **Aggregation** (allowing a relationship set to participate in another relationship set).

Short Answer	1-2 Lines Explanation	Source
--------------	-----------------------	--------

Short Answer	1-2 Lines Explanation	Source
What is DDL?	Data Definition Language is used to specify and modify the structures of database objects like tables and views (e.g., CREATE, ALTER, DROP).	
What is Data Abstraction?	The process of retrieving only required information and hiding background details, typically provided through Physical, Logical, and View levels.	
What is Data Independence?	The ability to change the data or structure at one level (e.g., physical) without affecting the schema definition or application programs at another level.	
What is an Entity Set?	A collection of entities of the same type, where each entity is distinguishable from others (e.g., all students in a class).	
What is Aggregation?	An E-R feature allowing an entire relationship set (along with its participating entity sets) to be treated as a single higher-level entity set that participates in another relationship.	

Unit II: Chapter 2 & 3 – Structured Query Language (SQL) and PL/SQL

This unit details the Structured Query Language (SQL) for relational interaction and PL/SQL for procedural extensions.

Important Points to Remember:

1. **SQL Overview:** SQL is the standard, case-insensitive language for Relational Database Management Systems (RDBMS). It is used to create, update, delete, and retrieve data.
2. **SQL Categories:** Commands fall under DDL (schema modification), DML (data manipulation), DCL (access control/authorization), and TCL (Transaction Control Language, for saving/undoing transactions like COMMIT/ROLLBACK).
3. **DML Core Queries:** **SELECT** is used to fetch data. **WHERE** specifies conditions. **ORDER BY** sorts results (ASC/DESC). **GROUP BY** groups rows with identical values, often used with aggregate functions. **HAVING** filters groups created by GROUP BY.
4. **SQL Constraints:** Rules defined on data columns including **PRIMARY KEY** (unique identifier, cannot be NULL), **FOREIGN KEY** (links two tables, ensuring referential integrity), **UNIQUE**, **NOT NULL**, and **CHECK** (limits value range).
5. **Indexes and Aggregate Functions:** An **Index** is a special data structure acting as a lookup table to accelerate data retrieval. **Aggregate Functions** (AVG, MIN, MAX, SUM, COUNT) perform calculations on a set of values to return a single result.
6. **Joins and Views:** **Joins** (INNER, LEFT, RIGHT, FULL) combine data from two or more tables. **Views** are virtual tables defined by a SELECT statement, used for abstraction or security, though not all views are updatable.

7. **PL/SQL Structure:** PL/SQL extends SQL with procedural capabilities and is organized into blocks (anonymous or named: Procedures/Functions) containing optional declaration and exception sections, and a mandatory executable section (BEGIN-END).
8. **Procedures and Functions:** A **Stored Procedure** is a named block that executes a sequence of statements but does not return a value directly. A **Function** is a named block that takes parameters and is required to return a single value.
9. **Cursors:** Cursors manage the memory area (context area) used during SQL statement processing, enabling the fetching and processing of rows one at a time. Cursors can be **Implicit** (system-managed) or **Explicit** (user-defined for multi-row results).
10. **Triggers and Exceptions:** **Triggers** are database objects that automatically execute specific actions (Event-Condition-Action) upon certain events (INSERT, UPDATE, DELETE). **Exceptions** handle runtime errors in the PL/SQL block (System-Defined or User-Defined).

Short Answer	1-2 Lines Explanation	Source
What is an Index?	A special data structure providing a fast access path to data records by storing indexed field values outside the main table.	
What is a View?	A virtual table derived from one or more base tables, used to present an abstract, partial view of the data or enforce security.	
What is a Stored Procedure?	A named PL/SQL subprogram that executes a group of statements to perform tasks but does not return a value directly.	
What is an Explicit Cursor?	A user-defined pointer declared in PL/SQL to manage the context area for a SELECT statement that returns multiple rows, allowing row-by-row processing.	
What is a Trigger?	A database object that automatically executes a predefined action (Event-Condition-Action model) when a specified DML operation occurs on a table.	

Unit III: Chapter 4 – Relational Database Design

This chapter details the theoretical foundation of the relational model, integrity constraints, and techniques for designing efficient databases (Normalization).

Important Points to Remember:

1. **Relational Model Concepts:** Key terms include **Relation/Table** (collection of data in rows/columns), **Tuple/Row** (single entry), **Attribute/Column** (data field), **Degree** (number of columns), and **Cardinality** (number of rows).
2. **Codd's Rules:** Codd proposed 13 rules (popularly 12) for a true RDBMS, emphasizing integrity, data independence (Physical and Logical), comprehensive data sublanguage, and relying only on

relational capabilities.

3. **Keys:** A **Super Key** is any set of attributes that uniquely identifies a record. A **Candidate Key** is a minimal super key. The **Primary Key** is the candidate key chosen by the designer. A **Foreign Key** links tables by referring to a primary key in another table.
4. **Integrity Constraints:** **Entity Integrity** requires the primary key value not to be NULL. **Referential Integrity** requires that any foreign key value must match an existing primary key value in the referenced parent table. **Enterprise Constraints** are additional, user-specified business rules, often spanning multiple tables.
5. **Design Issues:** Poor relational design is characterized by **Data Redundancy** and resulting anomalies: **Update anomalies** (inconsistency if redundant data is partially updated), **Insertion anomalies** (cannot insert partial data without related info), and **Deletion anomalies** (loss of necessary information when deleting a record).
6. **Normalization:** A process of reorganizing the database to minimize redundancy and ensure logical dependencies.
7. **Functional Dependency (FD):** A constraint $X \rightarrow Y$ holds if attribute set X uniquely determines attribute set Y.
8. **Normal Forms:**
 - **1NF (First Normal Form):** Requires all attributes to have atomic domains (no multivalued attributes).
 - **2NF (Second Normal Form):** Requires 1NF + elimination of **Partial Dependency** (no non-prime attribute depends on only a *part* of a candidate key).
 - **3NF (Third Normal Form):** Requires 2NF + elimination of **Transitive Dependency** (no non-prime attribute depends on another non-prime attribute).
 - **BCNF (Boyce-Codd Normal Form):** A stricter version of 3NF. For every non-trivial FD ($X \rightarrow Y$), X must be a **Super Key**.
9. **Decomposition Properties:** **Lossless Join** ensures that the original relation can be fully reconstructed by joining the decomposed tables. **Dependency Preservation** ensures that the functional dependencies on the original table can be verified by checking constraints only on the decomposed relations.

Short Answer	1-2 Lines Explanation	Source
What is Normalization?	The process of restructuring database tables to eliminate data redundancy and ensure logical dependencies, improving data integrity and consistency.	
What is Functional Dependency (FD)?	A constraint $A \rightarrow B$ means that the value of attribute set A uniquely determines the value of attribute set B.	
What is 2NF?	The Second Normal Form requires a table to be in 1NF and prevents partial functional dependencies (where a non-key attribute relies only on part of the composite key).	

Short Answer	1-2 Lines Explanation	Source
What is 3NF?	The Third Normal Form requires a table to be in 2NF and prevents transitive dependencies (where a non-key attribute depends on another non-key attribute).	
What is BCNF?	Boyce-Codd Normal Form is a higher form requiring that for every functional dependency $X \rightarrow Y$, X must be a super key for the relation.	

Unit IV: Chapter 5 – Database Transaction Management

This chapter focuses on managing simultaneous transactions to ensure data integrity and providing mechanisms for system recovery after failures.

Important Points to Remember:

- Transaction Definition and States:** A transaction is a logical unit of work. Transactions pass through states: **Active** (initial), **Partially Committed** (final operation executed), **Failed** (checks fail), **Aborted** (rolled back to previous consistent state), and **Committed** (successful completion).
- ACID Properties:** Essential properties guaranteeing transaction reliability: **Atomicity** (all operations succeed or none do), **Consistency** (database remains in a valid state), **Isolation** (concurrent transactions appear to run sequentially), and **Durability** (committed changes persist despite failure).
- Schedules and Serializability:** A **Schedule** is the chronological order of operations from concurrent transactions. **Serializability** is a correctness criterion ensuring that a non-serial schedule produces the same outcome as some serial schedule.
- Conflict Serializability:** Achieved if a schedule's precedence graph (built based on conflicting operations: Read-Write, Write-Read, Write-Write on the same data item by different transactions) is acyclic.
- Recoverability:** A schedule is **Recoverable** if a transaction commits only after all other transactions whose results it read have also committed. A **Cascadeless Schedule** prevents cascading rollbacks by ensuring no transaction reads data written by an uncommitted transaction.
- Concurrency Control:** Mechanisms used to control simultaneous execution to maintain isolation, often addressing problems like the **Lost Update Problem** and **Dirty Read**.
- Lock-based Protocols:** Use **Shared locks (S)** for reading and **Exclusive locks (X)** for reading/writing.
- Two-Phase Locking (2PL):** Guarantees conflict serializability. Transactions have a **Growing Phase** (acquiring locks, no release) followed by a **Shrinking Phase** (releasing locks, no acquisition). 2PL can lead to deadlocks.
- Deadlock Handling:** Deadlocks (circular waiting for resources) are managed via **Deadlock Prevention** (e.g., Wait-Die, Wound-Wait policies) or **Deadlock Detection** (using a Wait-For Graph to

find cycles).

10. **Time-stamp based Protocol:** Ensures serializability by assigning a unique timestamp (TS) to each transaction and using Read Timestamp (RTS) and Write Timestamp (WTS) on data items to order operations.
11. **Recovery Methods:** Methods to restore the database to a consistent state, including **Shadow-Paging** (uses a shadow directory/page table, requires no UNDO/REDO) and **Log-based Recovery** (maintains a log of modifications).
12. **Log-based Techniques: Deferred Database Modification** (NO UNDO/REDO) updates the physical database only upon transaction commit. **Immediate Database Modification** (UNDO/REDO) updates the database immediately, potentially requiring rollback operations if a failure occurs before commit. **Checkpoints** temporarily halt transactions and force write buffers to disk to minimize log checking during recovery.

Short Answer	1-2 Lines Explanation	Source
What is Atomicity?	The ACID property stating that a transaction must be completed fully, acting as a single logical unit; it either completes all steps or none at all.	
What is a Schedule?	The chronological sequence of operations from multiple transactions executing concurrently in a database system.	
What is Two-Phase Locking (2PL)?	A concurrency control protocol that enforces a growing phase (acquiring locks) followed by a shrinking phase (releasing locks) to ensure serializability.	
What is Deadlock?	A concurrency problem where a set of transactions is waiting for locks held by other transactions within the same set, leading to a standstill.	
What is Deferred Database Modification?	A log-based recovery technique (NO UNDO/REDO) where physical database updates are delayed until the transaction reaches its commit point.	

Unit V: Chapter 6 – NoSQL Databases

This unit introduces distributed systems and modern non-relational databases (NoSQL) designed for web-scale applications and Big Data.

Important Points to Remember:

1. **Distributed Database System (DDBS):** Consists of loosely coupled, geographically dispersed sites, offering fast processing, high reliability, and scalability.
2. **CAP Theorem:** Also known as Brewer's theorem, it states that a distributed data store cannot simultaneously guarantee **Consistency** (all reads see the latest write), **Availability** (system always responds), and **Partition Tolerance** (system operates despite network failures).

3. **Data Types: Structured data** (fixed schema, tabular), **Unstructured data** (no predefined organization, e.g., emails, images), and **Semi-structured data** (mix of both, uses tags/elements, e.g., XML, JSON).
4. **NoSQL Databases:** Stands for "Not only SQL." These are non-tabular database systems, popular for handling Big Data, supporting flexible or schema-free structures, and offering high horizontal scalability.
5. **Types of NoSQL:**
 - **Key-value store:** Simplest form, key is unique, value can be JSON, string, or binary object.
 - **Document store:** Stores data in documents (XML or JSON format), allowing flexible fields (e.g., MongoDB).
 - **Graph databases:** Used to store information about networks and relationships (e.g., social connections)[744, Q.11].
 - **Wide Column store:** Databases where the number of columns is not fixed for each record.
6. **BASE Properties:** Properties favored by many NoSQL systems, contrasting with ACID: **Basically Available**, **Soft State** (state can change without input), and **Eventual Consistency** (system consistency achieved over time).
7. **MongoDB:** A popular open-source, document-based NoSQL database. It uses a dynamic schema and provides high availability through **Replication** (creating copies across servers) and horizontal scalability through **Sharding** (splitting large datasets across multiple instances).
8. **MongoDB Operations:** Supports **CRUD** operations (Create/Insert, Read/Find, Update, Delete/Remove). **Indexing** is used for efficient query execution. **Aggregation** processes data using a pipeline (\$match, \$group, \$sort) to produce computed results. **MapReduce** is a programming model for batch processing and generating aggregate results over large data volumes.

Short Answer	1-2 Lines Explanation	Source
What is the CAP Theorem?	It posits that a distributed system cannot guarantee Consistency, Availability, and Partition Tolerance simultaneously; systems must choose two.	
What are BASE Properties?	Characteristics of NoSQL systems: Basically Available, Soft State, and Eventual Consistency, prioritizing availability over immediate consistency.	
What is a Document Store?	A type of NoSQL database that stores and retrieves data in semi-structured document formats like XML or JSON, offering flexible schemas (e.g., MongoDB).	
What is Sharding?	A MongoDB concept for horizontal scaling where large data sets are split into smaller subsets (shards) across multiple MongoDB instances.	
What are MongoDB's CRUD operations?	The basic database operations: C reate (Insert), R ead (Find), U pdate, and D eleate (Remove) documents.	

Unit VI: Chapter 7 – Advances in Databases

This chapter explores several contemporary and emerging database technologies and complex data handling formats.

Important Points to Remember:

1. **Active Databases:** Utilize the **Event-Condition-Action (ECA)** model, embedding triggers (active rules) to automatically react to database update operations.
2. **Deductive Databases:** Systems that make deductions based on **rules and facts** stored in the database, using concepts derived from logic programming (like Prolog).
3. **Main Memory Databases (MMDB):** Data resides permanently in the faster main physical memory, improving transaction speed, though a disk backup is required for failure recovery.
4. **Semantic Databases:** Systems that store the *meaning* of information (facts about objects), arranging data logically to ease application development and maintenance.
5. **Semi-Structured Data:** Data that lacks a fixed relational schema but possesses some structural properties, often managed using formats like XML.
6. **JSON (JavaScript Object Notation):** A text-based, lightweight, language-independent data interchange format built on **name/value pairs** and **ordered lists** (arrays).
7. **XML (eXtensible Markup Language):** A scripting language designed to **transport and store data** using custom, user-defined tags (unlike HTML, which focuses on displaying data).
8. **Document Type Definition (DTD):** Used to define the structural components (elements, attributes) of an XML document.
9. **Object Orientation (OODBMS/ORDBMS): Object-Relational Database Systems (ORDBMS)** integrate object-oriented features with relational models, serving as a migration path for relational users.
10. **Object-Relational Mapping (ORM):** An approach that maps tuples in database relations to objects in a programming language, simplifying object persistence and retrieval.
11. **Table Inheritance:** A feature where sub-tables inherit the schema (attributes) of a parent table in the relational database.
12. **Spatial Data:** Data related to space, representing the location, size, and shape of objects. It includes **Geometric Data** (points, lines, polygons) and **Geographic Data** (maps, satellite images, GPS information).

Short Answer	1-2 Lines Explanation	Source
What is an Active Database?	A database system using triggers defined by the Event-Condition-Action (ECA) model to automatically respond to database changes.	
What is JSON?	JavaScript Object Notation , a lightweight, text-based data interchange format built on name/value pairs and ordered lists.	

Short Answer	1-2 Lines Explanation	Source
What is XML?	eXtensible Markup Language , a scripting language using custom tags, designed primarily for the transport and storage of data.	
What is ORM?	Object-Relational Mapping is a technique defining how objects in a programming language correspond to tuples (rows) in relational database tables.	
What is Spatial Data?	Data related to physical space, encompassing both geometric constructs (shapes) and geographic information (locations, maps, GPS).	